
TinkerWithMe — Arduino Robotics & Electronics

Lesson Plan: From Touch to Trigger — Building a Motion-Activated Alarm System

Age Group: 10-12 years | **Duration:** 90 minutes | **Group Size:** Up to 12 students (pairs recommended)

1. Session Overview

In this session, students progress from a simple **capacitive touch sensor** circuit into a more advanced **motion-activated alarm system**. The lesson is structured as a scaffolded build: students first master digital input/output logic using a beginner-friendly touch module, then apply that same core skill (reading a sensor and reacting with outputs) to build a PIR motion sensor alarm that triggers a buzzer and LED.

This matters because it mirrors real-world security and safety systems — the same logic used in home alarms, automatic lights, and safety devices. Students leave with a working alarm prototype they built and understand, not just assembled.

2. Learning Objectives

By the end of this session, students will be able to:

- **Remember/Identify** the function of a PIR motion sensor, buzzer, and capacitive touch module in a circuit.

- **Understand** the difference between a momentary input (touch) and a continuous input (motion) and explain how each affects code logic.
- **Apply** digitalRead() and digitalWrite() functions correctly to wire and program two different sensor-based circuits.
- **Analyze** a non-working circuit or sketch to identify whether the fault is electrical (wiring) or logical (code).
- **Create** a fully functioning motion-activated alarm system that combines sensor input with dual outputs (sound + light).

3. Materials & Setup

Per Pair of Students (recommended grouping)

ITEM	SPECIFICATION	QUANTITY
Arduino Board	Arduino Uno R3 (or compatible clone)	1
USB Cable	USB Type-A to Type-B	1
Breadboard	830-point solderless breadboard	1
Capacitive Touch Sensor Module	TTP223 digital touch module	1
PIR Motion Sensor	HC-SR501 with adjustable sensitivity & delay	1
Buzzer	5V active piezo buzzer	1
LED	5mm Red LED	1
Resistor	220 ohm, 1/4 watt	1
Jumper Wires	Male-to-Male, assorted lengths	15-20
Jumper Wires	Male-to-Female (for sensor modules)	6
Laptop/Computer	With Arduino IDE (version 1.8+ or 2.x) pre-installed	1

Shared/Classroom Materials

- Spare Arduino Uno boards (2-3) for quick swaps if a board fails
- Spare LEDs, resistors, and buzzers (small parts tray)
- Multimeter (1-2, for troubleshooting demonstrations)
- Whiteboard/projector for circuit diagrams and code walkthrough
- Printed circuit diagrams (1 per pair) as backup for visual learners

Setup Instructions (Before Students Arrive)

- Test one full alarm circuit end-to-end to confirm code and wiring work with your specific PIR batch (sensitivity varies by unit).
- Pre-load Arduino IDE with correct board/COM port settings on all laptops.
- Arrange desks in pairs with shared component trays to reduce setup time.
- Have both code sketches saved on a shared drive or USB stick as backup.

4. Minute-by-Minute Schedule

TIME	DURATION	ACTIVITY
0:00–0:05	5 min	Welcome, quick recap of digital inputs/outputs, introduce today's goal (build an alarm system)
0:05–0:10	5 min	Safety briefing: handling components, static precautions, powering down before rewiring
0:10–0:30	20 min	Project 1: Capacitive Touch Sensor — wiring, uploading code, testing toggle behavior
0:30–0:35	5 min	Quick group discussion: "What did the touch sensor teach us about inputs?" Bridge to motion sensing
0:35–0:40	5 min	Stretch/Break — short movement break, redistribute alarm system components
0:40–0:55	15 min	Project 2: Alarm System — wiring PIR sensor, buzzer, and LED on breadboard
0:55–1:10	15 min	Upload alarm code, calibrate PIR sensor, test motion detection
1:10–1:20	10 min	Debugging round — instructor circulates, pairs fix issues, fine-tune sensitivity/delay
1:20–1:30	10 min	Live demo showcase, class discussion, wrap-up, and take-home challenge briefing

5. Project Breakdowns

Project 1: Capacitive Touch Sensor (Beginner) — 20 min allocated

Note: Original project time is 60 minutes; condensed here to 20 minutes as a warm-up. Skip the extension challenge in this project if time is tight — save it for early finishers.

Step-by-Step Instructions

- Place the TTP223 touch module on the breadboard.
- Connect module VCC to Arduino 5V, GND to Arduino GND.
- Connect module signal (SIG/OUT) pin to Arduino Digital Pin 2.
- Connect the built-in LED (Pin 13) — no external LED needed for this project, but pairs may wire an external LED with a 220 ohm resistor to Pin 13 for a bigger visual effect.
- Open Arduino IDE, select correct Board (Arduino Uno) and Port.
- Type or paste the code below, upload it.
- Touch the sensor pad — LED should toggle ON, touch again to toggle OFF.

Full Working Code

```
// -----  
// Project: Capacitive Touch Sensor LED Toggle  
// Description: Each touch toggles the LED ON/OFF  
// -----  
  
const int touchPin = 2;      // TTP223 signal pin connected here  
const int ledPin = 13;      // Built-in LED pin  
  
bool ledState = false;      // Tracks current LED state (on/off)  
bool lastTouchState = LOW;  // Remembers previous touch reading  
  
void setup() {  
  pinMode(touchPin, INPUT); // Touch sensor as input  
  pinMode(ledPin, OUTPUT);  // LED as output  
  Serial.begin(9600);       // For debugging in Serial Monitor  
}  
  
void loop() {  
  bool touchState = digitalRead(touchPin); // Read current touch state  
  
  // Detect a NEW touch (transition from not-touched to touched)  
  if (touchState == HIGH && lastTouchState == LOW) {  
    ledState = !ledState;      // Flip the LED state  
    digitalWrite(ledPin, ledState); // Apply new state to LED  
    Serial.println(ledState ? "LED ON" : "LED OFF");  
    delay(50); // Small debounce delay to avoid double-triggering  
  }  
  
  lastTouchState = touchState; // Save reading for next loop comparison  
}
```

Common Mistakes & Fixes

- **LED does not toggle:** Check that the touch module's signal pin matches the code's `touchPin` (Pin 2). Confirm VCC/GND are not swapped.

- **LED flickers rapidly on single touch:** Increase the debounce delay from 50 to 100–150 milliseconds.
- **Sensor always reads HIGH:** Some TTP223 modules default to "toggle mode" via a solder jumper — check module documentation; may need to switch to "direct mode."
- **Nothing happens at all:** Confirm the correct board and port are selected in the Arduino IDE before uploading.

Success Criteria

- LED reliably toggles ON and OFF with each distinct touch.
- Student can explain, in their own words, what "toggle" means in code.
- Student can identify the three wires (VCC, GND, Signal) and their purpose.

Project 2: Basic Alarm System (Intermediate) — 40 min allocated

Step-by-Step Instructions

- Remove the touch sensor from the breadboard (keep wiring area clear).
- Place the PIR motion sensor module on or beside the breadboard.
- Connect PIR VCC to Arduino 5V, GND to Arduino GND, OUT to Digital Pin 7.
- Insert the LED into the breadboard; connect the long leg (anode) through a 220 ohm resistor to Digital Pin 13; connect the short leg (cathode) to GND.
- Connect the buzzer: positive leg to Digital Pin 8, negative leg to GND.
- Double-check all GND connections share a common ground rail.
- Upload the alarm code below.
- Wait for the sensor calibration countdown in the Serial Monitor (about 20 seconds) — this lets the PIR sensor stabilize.
- Once ready, wave a hand in front of the PIR sensor to trigger the alarm (LED + buzzer).
- Adjust the two potentiometers on the PIR module (sensitivity and time delay) if the alarm is too sensitive or not sensitive enough.

Full Working Code

```
// -----
// Project: Basic Motion-Activated Alarm System
// Description: PIR sensor detects motion and
// triggers a buzzer + LED as an alarm
```

```
// -----

const int pirPin = 7;          // PIR sensor OUT pin
const int buzzerPin = 8;      // Buzzer positive pin
const int ledPin = 13;        // Alarm LED pin

int calibrationTime = 20;     // Seconds to let PIR sensor stabilize

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);
  Serial.println("Calibrating PIR sensor, please wait...");

  // Countdown so students see the sensor "warming up"
  for (int i = 0; i < calibrationTime; i++) {
    Serial.print(".");
    delay(1000);
  }

  Serial.println("");
  Serial.println("Sensor ready! Alarm system ARMED.");
}

void loop() {
  int motionState = digitalRead(pirPin); // Read PIR sensor state

  if (motionState == HIGH) {
    // Motion detected – activate alarm outputs
    digitalWrite(ledPin, HIGH);
    digitalWrite(buzzerPin, HIGH);
    Serial.println("Motion detected! ALARM TRIGGERED!");
  } else {
    // No motion – keep alarm silent and LED off
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);
  }

  delay(100); // Small delay to stabilize readings
}
```

Common Mistakes & Fixes

- **Alarm triggers constantly, even with no motion:** The PIR sensor may not have finished calibrating — wait the full 20 seconds. Also check for nearby moving air (fans) or direct sunlight, which can cause false triggers.
- **No response at all:** Verify OUT pin from the PIR module matches Pin 7 in code. Confirm PIR is receiving 5V (check with multimeter if unsure).
- **Buzzer stays silent but LED works:** Buzzer polarity may be reversed, or it is a passive buzzer requiring PWM (`tone()`) rather than `digitalWrite()`. Confirm you have an *active* buzzer.

- **LED very dim or not lighting:** Check resistor is in series correctly and LED orientation (long leg = positive).
- **Alarm won't turn off after motion stops:** This is expected PIR behavior — most modules hold HIGH for several seconds after detecting motion. Adjust the delay potentiometer on the sensor to shorten this.

Success Criteria

- LED and buzzer activate simultaneously within 1–2 seconds of detected motion.
- Alarm resets (LED/buzzer off) when no motion is present.
- Student can explain the purpose of the calibration delay in the code.
- Student can locate and adjust the PIR sensitivity/delay potentiometers.

6. Assessment

- **Live Demo Challenge:** Each pair demonstrates their working alarm system to the class and explains, in their own words, what happens when motion is detected (checks Understanding & Application).
- **Rapid-Fire Quiz:** Instructor asks quick questions during debugging rounds — e.g., "What pin is the buzzer on?" "What does HIGH mean here?" — to check real-time comprehension.
- **Code Tracing Exercise:** Instructor covers the projector code and asks a student to predict what the LED will do at a specific line, testing logical understanding, not memorization.
- **Peer Teaching:** Pairs that finish early explain their wiring to a struggling pair — a strong indicator of mastery when done successfully.

7. Extensions & Challenges

- **Silent Mode Switch:** Add a push button that arms/disarms the alarm system, so it only triggers when "armed."
- **Flashing Alarm:** Modify the code so the LED blinks rapidly instead of staying solid when triggered (introduces basic timing with `millis()`).
- **Combine Both Projects:** Use the touch sensor as the "arm/disarm" switch for the alarm system instead of a physical button.
- **Serial Monitor Logger:** Add a counter that logs how many times motion was detected during the session, printed to Serial Monitor.

- **Multi-Zone Alarm:** For advanced learners with spare parts, wire a second PIR sensor to simulate a two-room security system.

8. Troubleshooting Guide (FAQ)

ISSUE	LIKELY CAUSE	FIX
Arduino IDE won't upload code	Wrong board/port selected, or USB cable is charge-only	Check Tools > Board and Port settings; swap USB cable
PIR sensor triggers alarm constantly	Sensor still warming up, or interference from movement/light	Wait full calibration period; reposition away from windows/fans
Buzzer makes no sound	Wrong buzzer type (passive vs active) or wiring reversed	Confirm active buzzer type; check polarity
Touch sensor unresponsive	Module in wrong mode (toggle vs momentary)	Check for solder jumper on module; consult module datasheet
Code uploads but nothing happens	Loose breadboard connections	Press wires firmly, check for bent pins, re-seat components
LED stays on permanently		